

The Design and Implementation of a Log-Structured File system

Mendel Rosenblum and John K. Ousterhout,
Electrical Engineering and Computer Sciences, Computer Science Division University of California

2026. 07. 28

Presentation by 김정민, 정다원

kyrios3929@dankook.ac.kr

defimedjustice1220@gmail.com

Contents

1. Abstract
2. Introduction
3. Design for file systems of the 1990's
4. Log-structured file systems
5. Crash recovery
6. Experience with the Sprite LFS
7. Conclusion

1. Abstract

- LFS는 log-like structure로 모든 수정을 sequentially하게 저장
- Sequential write를 통하여 seek 감소와 쓰기, crash recovery 속도 증가 효과 얻을 수 있다
- Inode, bitmap, directory 영역이 따로 존재하는 것이 아닌, 모든 것이 log 형태로 Disk에 append된다는 특징

1. Abstract

- 계속 append를 하다보면, 언젠간 Disk가 꽉 차는 문제가 발생
- segment cleaning을 수행해 빈 공간 확보
- 실제 실험 결과, 작은 파일을 write할 때 UNIX 대비 10배 이상 빠른 결과
 - Cleaning overhead를 포함하더라도 disk 대역폭의 70%는 실제 쓰기에 사용하는 것을 확인 (UNIX는 disk 대역폭의 5~10% 정도만 사용)

2. Introduction

- 현재 CPU의 속도 대비 Disk의 속도는 매우 느리다. 이로 인하여 CPU가 Disk 때문에 대기해야 하는 상황이 발생 → Disk-bound
- 시간이 지나면서 Memory의 크기는 증가
→ 실제 읽기는 RAM에서 수행되고 Disk의 주된 작업은 Write

Read는 캐시에서 처리하고 Write를 최대한 빠르게 만들자는 것

2. Introduction

- LFS의 log와 기존 log의 차이점

→ 기존의 log는 먼저 log에 쓴 후 원래 위치로 다시 복사하는 과정을 거쳐야하는 임시 저장소의 역할, LFS의 log는 log 자체가 영구 저장소

3. Design for systems of the 1990's

- File System은 technology (새 CPU, 메모리, Disk)와 workload (사용자가 무엇을 많이 하는지)에 의해 결정

→ 이 두 가지를 고려하여 File System을 설계해야 한다.

- File System Design에 영향을 주는 요인들 : Processor (CPU), Disks, Main memory

3.1 Technology

- Processor (CPU) : CPU의 속도는 점점 빨라지고 있지만, Disk의 속도는 여전히 느리기 때문에 CPU가 대기하는 시간이 발생
- Disk와 processor의 성능 차이가 점점 벌어진다
- Main memory : Memory의 크기가 점점 커지면서 읽기는 대부분 RAM에서, Disk는 write만을 주된 작업으로 수행

3.1 Technology

- Disk : Disk의 성능은 Transfer bandwidth (순차 읽기 속도)와 Access time (seek + Rotation delay)에 의해 결정된다.
- 즉 작은 I/O를 많이 할 수록 seek가 많이 일어나기 때문에 CPU가 빨라져도 성능 향상이 거의 나타나지 않는다

3.2 Workloads

- 평균적인 파일의 크기는 몇 KB 정도
 - Disk에서는 4KB → write → 4KB → write ...과 같은 비효율적 접근을 수행
 - 파일 한 개 생성 시, data, inode, directory, metadata를 모두 수정해야 한다
- 작은 파일의 경우에는 Data보다 Metadata를 더 많이 수정하는 상황이 발생
(단, 큰 파일의 경우에는 순차 읽기를 수행해서 seek가 거의 발생 X)

3.3 Problems with existing file systems

- 현재의 File System에는 2가지의 문제점이 존재

1. Data가 Disk 곳곳에 흩어져있음

1. Synchronous (동기식) 쓰기를 많이 수행

3.3 Problems with existing file systems

Data를 Disk 곳곳에 흩어놓는 문제점

- File 1개를 생성하더라도 inode, bitmap, directory 등 여기저기에 데이터를 저장하기 때문에 seek가 많이 발생
- 작은 파일의 경우 seek time이 전체 실행 시간의 대부분을 차지하는 비효율적인 상황이 발생

3.3 Problems with existing file systems

Synchronous (동기식) 쓰기를 많이 수행하는 문제점

Synchronous write : 쓰기 요청이 들어오면 disk에 기록될 때까지 CPU는 대기 상태에 있다가 다 써지면 그 후에 다음 작업을 수행하는 방식

→ 안전하지만 CPU가 낭비

Asynchronous write : 쓰기 요청시 RAM에 잠시 저장했다가 어느 정도 모이면 disk에 한 번에 write하는 방식

→ CPU의 대기 시간이 줄어들지만 위험성이 증가

3.3 Problems with existing file systems

Synchronous (동기식) 쓰기를 많이 수행하는 문제점

작은 파일의 경우 쓰려는 데이터보다 inode, directory, bitmap의 용량이 더 커져버리는 상황이 발생하여 CPU가 대기를 반복

→ CPU의 성능이 좋아져도 disk가 느리면 CPU가 대기를 반복

4. Log-structured file systems : Related Work

Logging : 기존의 log는 복구를 위한 보조 저장소였지만 LFS에서는 log가 FS 그 자체

Writing buffer : 기존에도 write buffer를 사용은 했지만, 결국 원래 위치에 다시 기록하는 과정이 필요

→ LFS는 log 끝에 데이터를 기록하면서 기존 write buffer보다 더 큰 순차 쓰기 효과

즉, 이 LFS 연구는 충분한 가치를 가진다.

4. Log-structured file systems

- 파일 데이터와 metadata 변경을 메모리의 파일 캐시에 모은 뒤, 이를 log 형태의 큰 순차 쓰기로 Disk에 append하는 방식

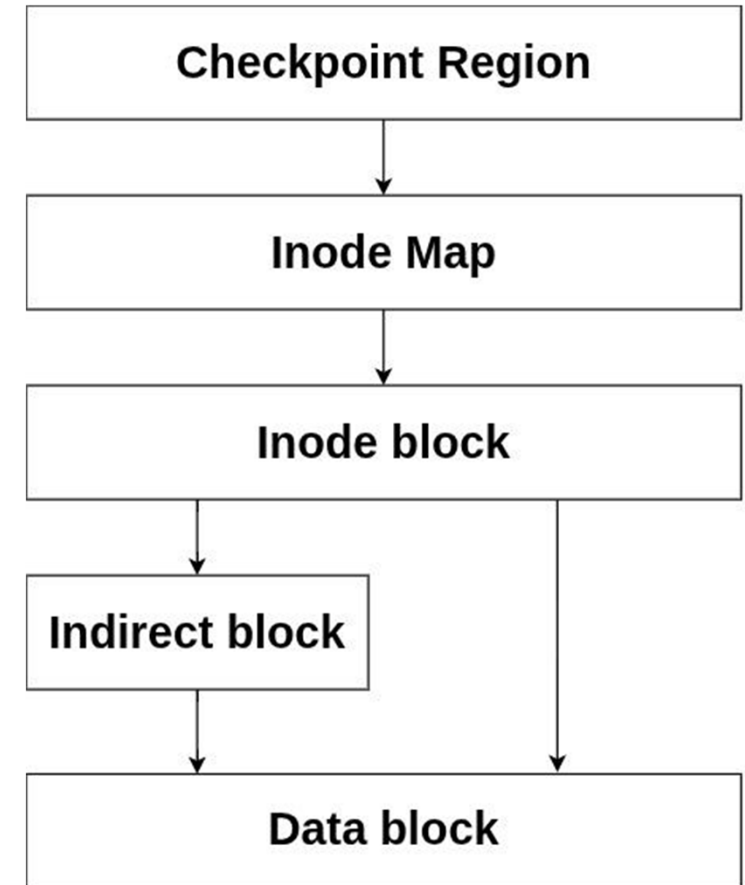
→ 즉 write buffer 역할

→ sequential write를 수행하기 때문에 seek가 없고, Disk 대역폭을 효율적으로 활용

4.1 File location and Reading

Inode 구조

- LFS 또한 FFS와 비슷하게
inode block → indirect block
→ data block 순으로 접근하는
inode 구조를 사용
- Inode 위치가 동적이라는 차이점



4.1 File location and Reading

그럼 그 inode (inode block)들을 어떻게 찾을까

- Inode를 찾기 위해 inode의 위치를 저장한 inode map을 사용
- checkpoint를 통해 inode map에 접근한다
- Inode의 번호를 읽은 후 inode map을 조회하여 현재 inode의 위치를 탐색

4.1 File location and Reading

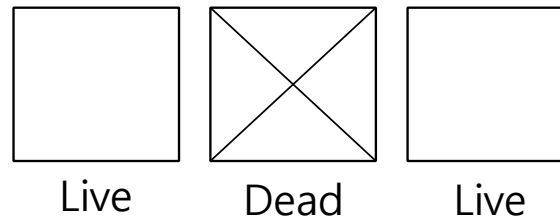
Inode map을 추가로 Read해야 하는데, 읽기 성능이 떨어지지 않나

- Inode map의 크기는 매우 작아서 대부분 Memory에 캐시되어 있기 때문에 추가적인 Disk 접근이 거의 없음

→ 따라서 읽기 성능 저하는 거의 일어나지 않는다

4.2 Free space management : segments

- 계속 append하다 보면 언젠간 Disk가 꽉 참
- 삭제나 수정을 통한 빈 공간이 존재는 하지만, 작고 분산되어 있기 때문에 활용 힘들다



- Block을 삭제하면, 그 자리에 데이터를 새로 넣기 위해 Random write를 진행

4.2 Free space management : segments

해결방법 1

- Thread : 빈 공간을 따라 새로운 데이터를 끼워넣는 방법
- 장점 : 추가적인 동작 없이 바로 쓸 수 있음
- 단점 : 파편화 된 공간에 쓰는 것은 Random write와 같은 상황

4.2 Free space management : segments

해결방법 2

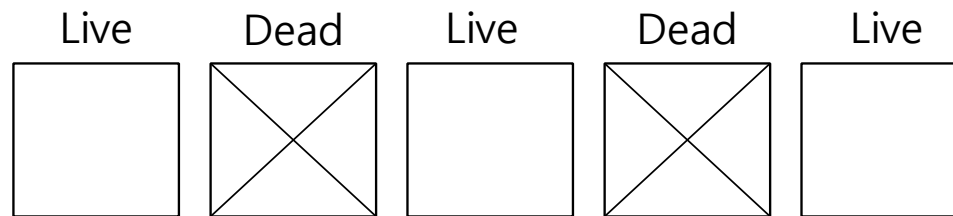
- Copy : 데이터를 복사해 새 공간에 옮겨서 빈 공간을 만드는 방법
- 장점 : LFS처럼 Sequential write 가능
- 단점 : 쓰기 증폭 발생

4.2 Free space management : segments

절충안 : Segment

- Segment는 disk가 순차쓰기를 하는 최소 단위
- Disk를 Segment 단위로 쪼갬 후, segment 단위로 Thread를 진행
- Segment 크기가 충분히 크면, 내부의 순차접근이 외부에서 segment 찾는 시간을 상회하여 Disk 대역폭을 효율적으로 활용 가능

4.3 Segment cleaning mechanism



- 수정을 반복하다 보면, 위와 같이 live block과 dead block이 섞이게 된다
→ Segment cleaning을 수행

Segment cleaning을 어떻게 수행할까

1. 일정량의 segment를 읽는다
2. 읽은 segment에서 Live block만 식별한다
3. 식별한 Live block를 새 segment에 쓴다
4. 그 후 원래 segment를 삭제한다

4.4 Segment cleaning policies

Segment cleaning에서 고려할점

- 언제 실행될 것인가?
- 얼마나 많은 segment를 정리할 것인가?
- 어떤 segment를 선택할 것인가?
- 블록을 어떻게 묶어야 할 것인가?

4.4 Segment cleaning policies

Sprite LFS는 다음과 같은 기준을 따른다.

실행 시작:

clean segment 개수가 **임계점** (몇십개)
이하로 떨어질때

실행 종료:

clean segment 개수가 **임계점** (50~100)
에 도달할 때까지

4.4 Segment cleaning policies

Greedy 정책

기준점: Write cost

새로 쓰는 Data에 비하여 Disk가 얼마나 바빠지는가?

$$\frac{\text{Disk I/O}}{\text{write new}} = \frac{\text{read segs} + \text{write lives} + \text{write new}}{\text{write new}} = \frac{N + N * u + N * (1 - u)}{N * (1 - u)} = \frac{2}{1 - u}$$

(N : segment 개수, u : utilization (live data의 비율))

Segment에서는 write cost가 다음과 같이 계산되며, cost가 제일 작은, 즉 utilization (live data의 비율)이 가장 작은 segment를 선택한다

4.4 Segment cleaning policies

Hot data와 cold data

- Cold data : 거의 수정되지 않는 데이터, 따라서 live block의 비율이 높다.
- Hot data : 자주 수정되는 데이터, 따라서 live block의 비율이 낮다.

가정 1 : greedy 정책은 utilization을 통해 자연스럽게 두 data를 분리할 수 있다.

4.4 Segment cleaning policies

선택된 segment의 utilization이 낮으면 낮을수록
복사해야할 live data의 양이 적어 cleaning overhead가 감소한다.

따라서 greedy 정책을 따랐을때,

**가정 2 : Workload 내부의 hot data와 cold data의 접근 패턴이 양극단에
몰릴수록 cleaning overhead는 감소한다.**

4.5 Simulation

이를 검증하기 위하여, 다음과 같은 2가지 Workload에서 실험을 진행

1. Uniform: 모든 파일이 동일한 빈도로 선택

1. Hot & Cold: 전체 파일중 10%에 해당하는 Hot file이 90% 확률로 선택됨

4.5 Simulation

실험환경

- disk 전체 utilization은 일정하게 유지하였다.
- 쓰기와 cleaning 비용만 고려하였다.

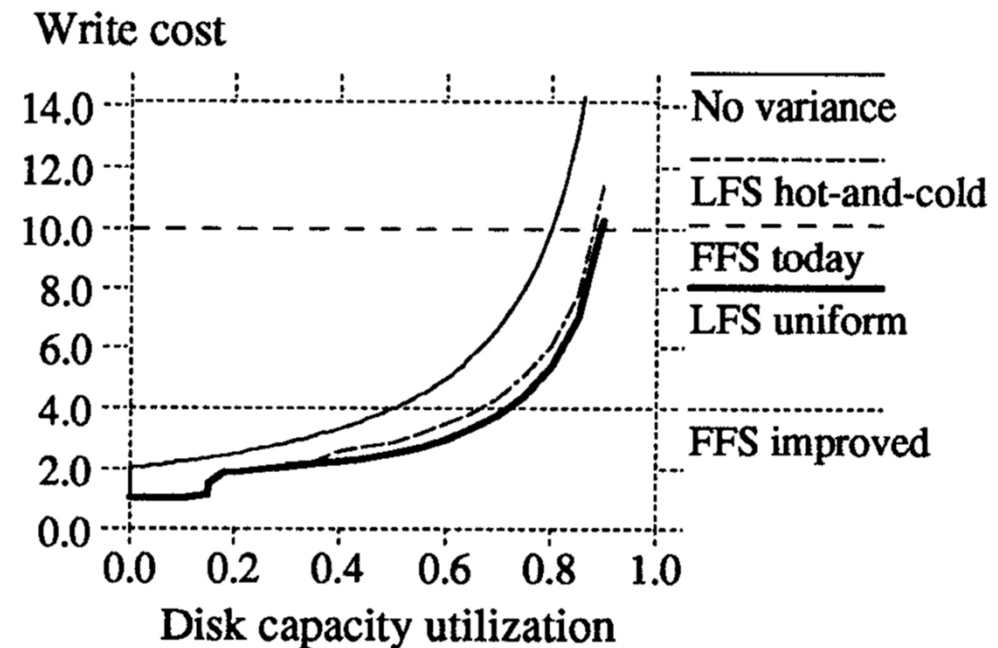
실험 과정

1. 빈 segment가 모두 소진될 때까지 write를 반복 수행
2. cleaner가 정해진 임계값만큼의 free segment를 확보
3. 위 과정을 반복

4.5 Simulation

실험결과

- Utilization을 기준으로 segment를 선택하는 정책은 예상보다 낮은 write cost를 보였다.
- 예상과 달리**, Hot and Cold workload에서 Uniform workload보다 더 높은 write cost가 측정되었다.

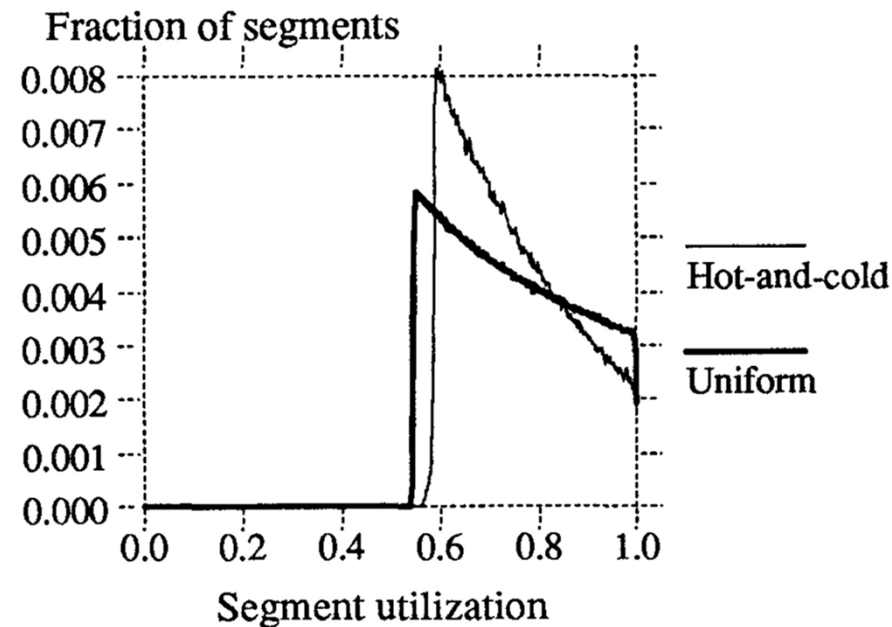


Greedy 정책의 한계

Hot data에서만 데이터 회수가 집중되기 때문에 cold data의 영역은 회수가 미뤄진다.

이에 따라, cold data가 상대적으로 많은 segment의 회수가 늦어지기 때문에 hot data와 cold data의 분리가 어려워진다.

미뤄진 segment를 회수하기 위해 segment의 age도 함께 고려해야한다.



단순 greedy 정책에서의 workload에 따른 utilization 분포도

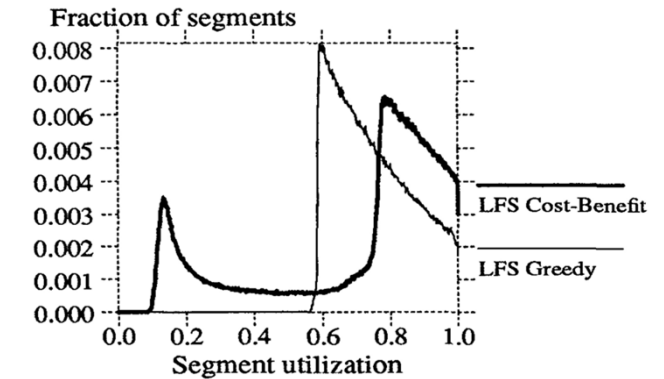
4.6 Segment cleaning policies

Cost-benefit policy:

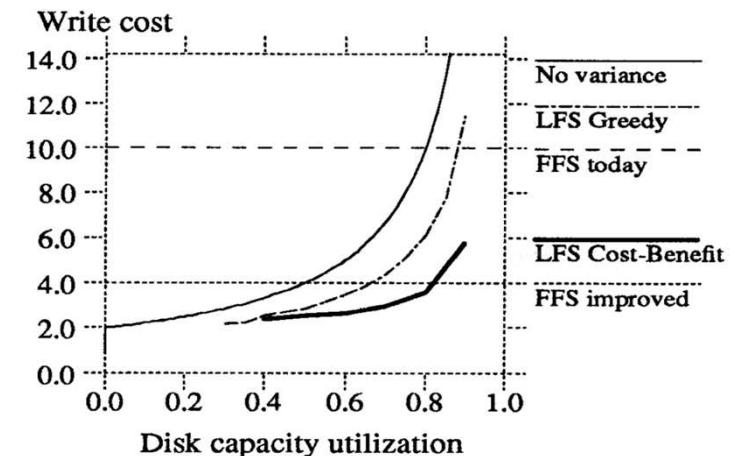
benefit / cost 가 최대가 되는 segment를 선택하는 정책

- cost: 얼마나 많은 data를 복사해야 하는가
- benefit: 빈 공간이 얼마나 생기는지
+ 해당 segment가 얼마나 오래됐는지

$$\frac{benefit}{cost} = \frac{(1 - u) * age}{1 + u}$$



Cost & Benefit 방법에서는 cold data와 hot data가 잘 분리된 것을 확인할 수 있다.



기존방법대비 write cost가 감소한 모습을 확인할 수 있다.

4.7 Segment usage table

- 각 segment의 상태 (segment, live bytes, age)를 저장해 놓은 table
- cleaning을 할 때마다 모든 segment를 읽으면 느리기 때문에 table만 보고 cost-benefit 식의 값을 계산하여 cleaning 대상을 빠르게 선택
- 파일이 삭제/갱신 될 때 live byte 수와 age 정보도 함께 갱신하여 평소에 최신 상태를 유지

5. Crash recovery

- LFS는 계속 새 위치에 데이터를 저장
→ 시스템이 갑자기 종료된다면 현재 FS의 상태를 어떻게 찾을까
- FFS에서는 inode가 항상 같은 위치에 존재하기 때문에 그 위치의 inode를 다시 읽으면 된다
- LFS는 새 위치에 계속해서 데이터를 작성하기 때문에 crash가 발생시
→ 이 inode가 최신인지, 최신 inode map이 어딘지 알 수 없음
- 따라서 checkpoint를 도입

5.1 Checkpoints

- FS의 snapshot
- Inode map의 위치가 저장되어 있기 때문에 재부팅 시 가장 먼저 찾아야 한다 → Disk의 고정된 위치에 저장
- Checkpoint를 작성하는 중에 crash가 발생하면 손상이 될 수 있기 때문에 항상 두 개의 checkpoint를 두고 사용

5.1 Checkpoints

- 모든 write 마다 checkpoint를 작성하면 성능이 저하
→ 일정 시간/FS가 종료될 때마다 기록하게 된다.
- Checkpoint 저장 → 파일 수정 → crash
→ 이런 경우 최신 데이터가 소실 가능
- 따라서 Roll-forward라는 방법을 추가한다.

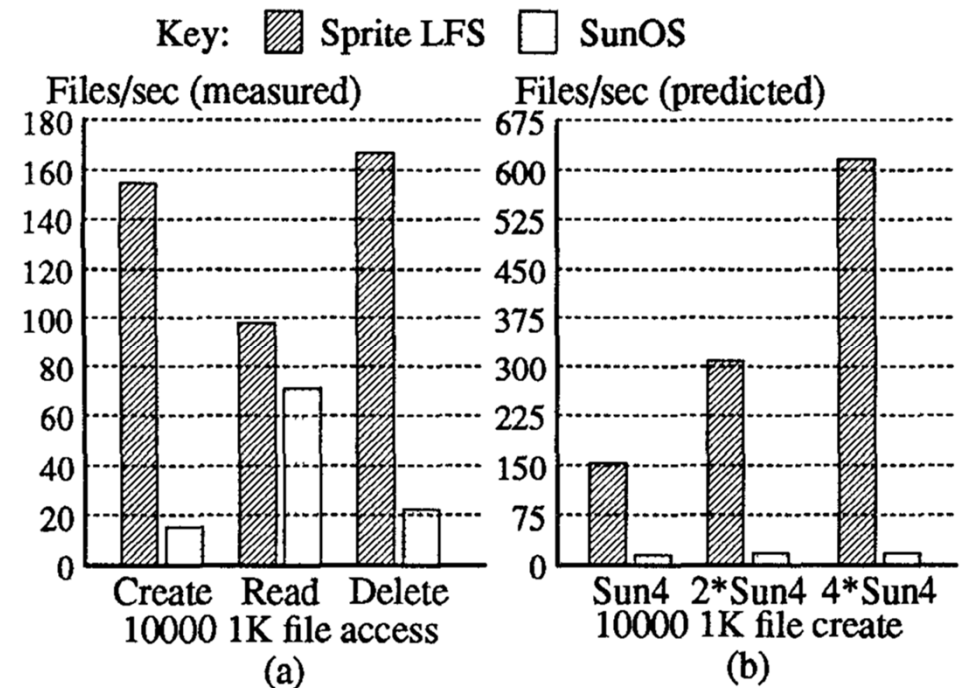
5.2 Roll-forward

- Checkpoint 이후에 기록된 log를 다시 따라가는 과정
- Checkpoint 이후의 segment를 읽어서 정상적으로 기록된 변경 사항 inode map에 반영한다.
- 데이터가 정상인지 확인하기 위해 segment의 inode, offset 등의 정보를 저장하고 있는 segment summary block을 사용
 - 먼저 segment summary block을 읽고 데이터가 정상이면 inode map을 갱신, 아니라면 그 segment는 복구하지 않는다

6. Experience with the Sprite LFS

FFS 대비 sprite LFS 실험 : 작은 파일/큰 파일 write/cleaning 비용, crash-recovery 시간/기타 오버헤드 비교

- **작은 파일 write :**
LFS가 FFS 대비 10배 정도 빠른 성능
→ seek가 거의 없기 때문
- **작은 파일 삭제 :**
FFS는 inode/directory/bitmap 등 여러 곳을 수정하지만,
LFS는 삭제할 정보도 순차 기록되어 있기 때문에 FFS보다 빠른 성능

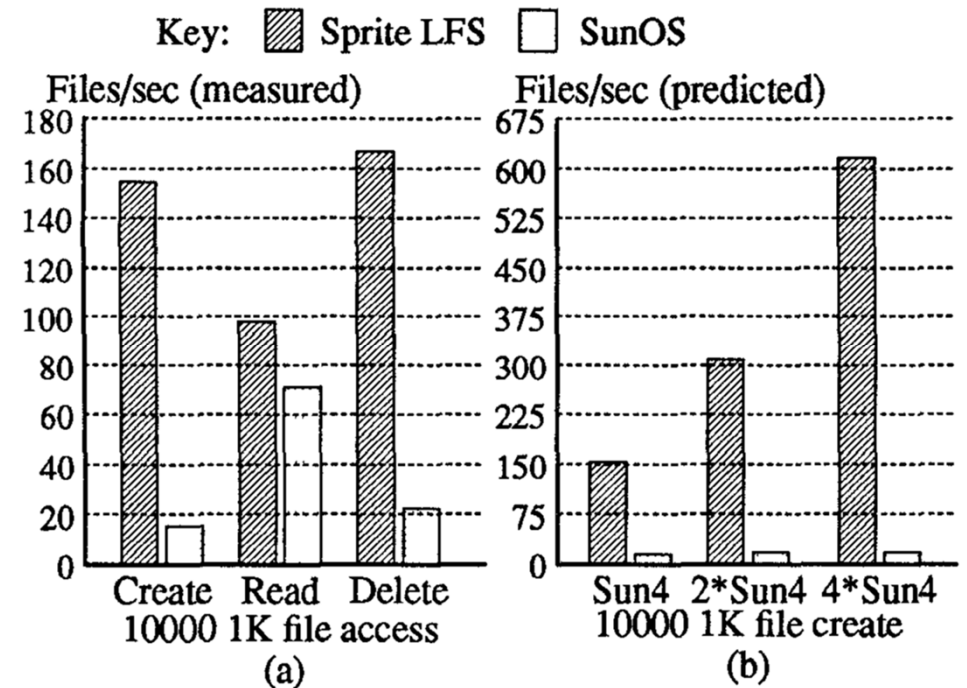


6. Experience with the Sprite LFS

- CPU 성능에 따른 차이 (오른쪽 그래프)

FFS는 disk가 핵심 병목이기 때문에 CPU에 따른 성능 차이가 보이지 않지만, LFS는 CPU에 따른 성능격차가 관측된다.

앞에서 언급했듯이 Disk와 CPU간 격차가 커질 것으로 예상되기에, LFS의 성능이 더 좋아질 것으로 예상된다.



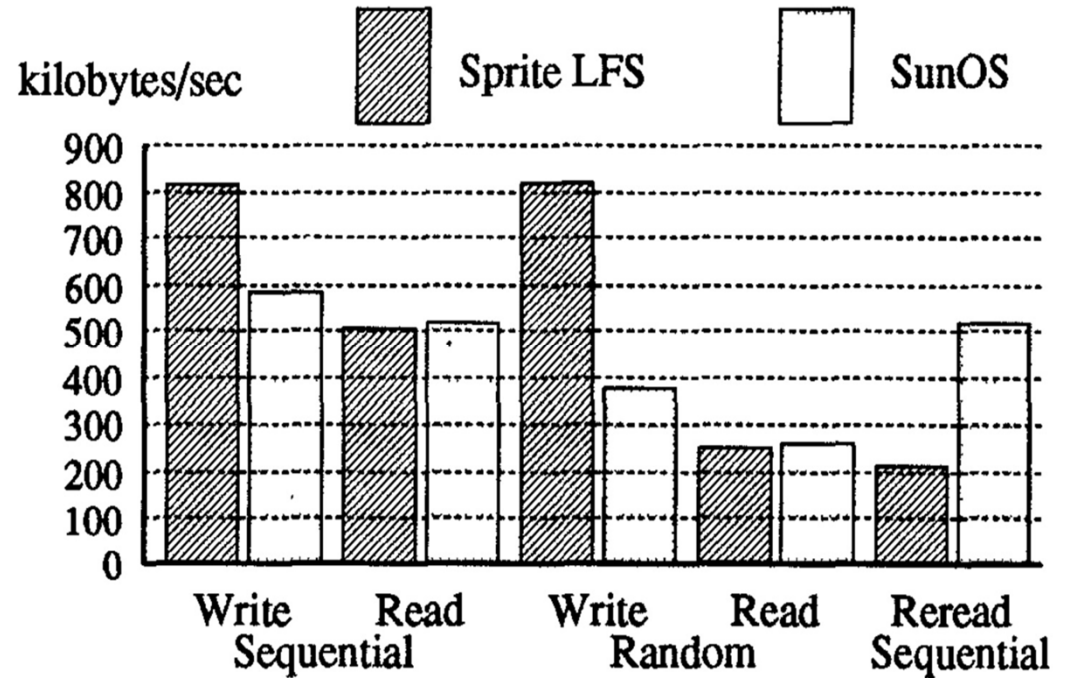
6.1 Micro-benchmarks

실험과정

1. 100MB 파일 순차 쓰기
2. 해당 파일 순차 읽기
3. 임의 위치에 100MB 랜덤 쓰기
4. 파일에서 100MB 랜덤 읽기
5. 파일 전체를 다시 순차 읽기

그래프

각 과정에서의 대역폭을 나타낸다.



6.1 Micro-benchmarks

- **Write :**

LFS의 순차쓰기로 인해, LFS는 양쪽 모두 높은 대역폭

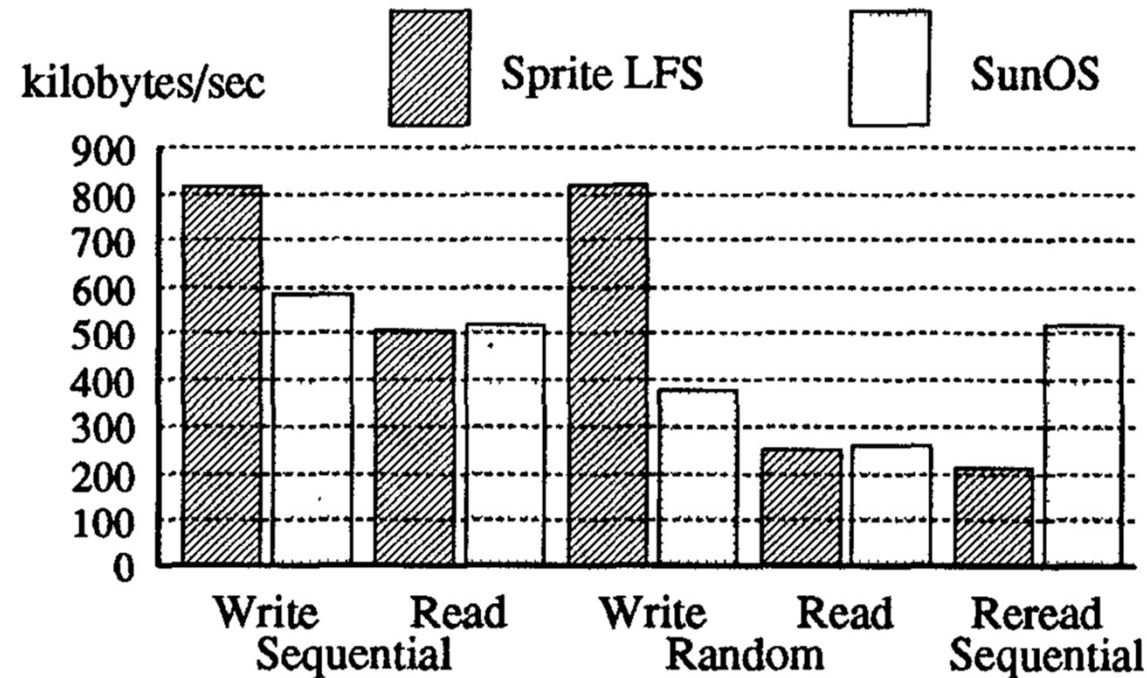
FFS는 랜덤 쓰기로 인한 디스크 탐색이 많이 발생해 성능이 크게 저하

- **Read :**

inode map이 Memory에 위치하기 때문에 LFS의 Read 성능이 저하 없음

따라서 FFS와 LFS의 성능이 비슷하게 나타난다

.



6.1 Micro-benchmarks

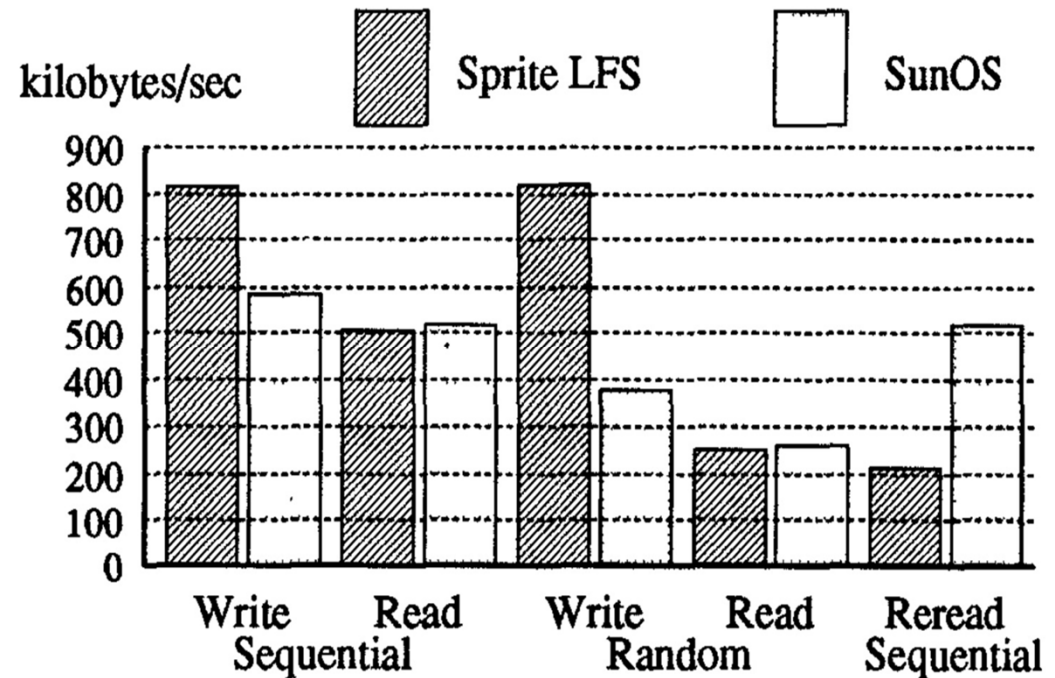
ReRead Sequence

: Random Write 후 순차적으로 읽은 결과

FFS가 더 좋게 나온 이유는 LFS와 FFS Locality 차이에 있다.

- FFS - 논리적 지역성 : 연관 있는 data끼리 뭉친다
- LFS - 시간적 지역성 : 비슷한 시기에 쓰인 data끼리 뭉친다

Random Write가 LFS의 시간적 지역성을 무너뜨렸기 때문에 다음 순차읽기에서 지역성을 활용할 수 없게 된다.



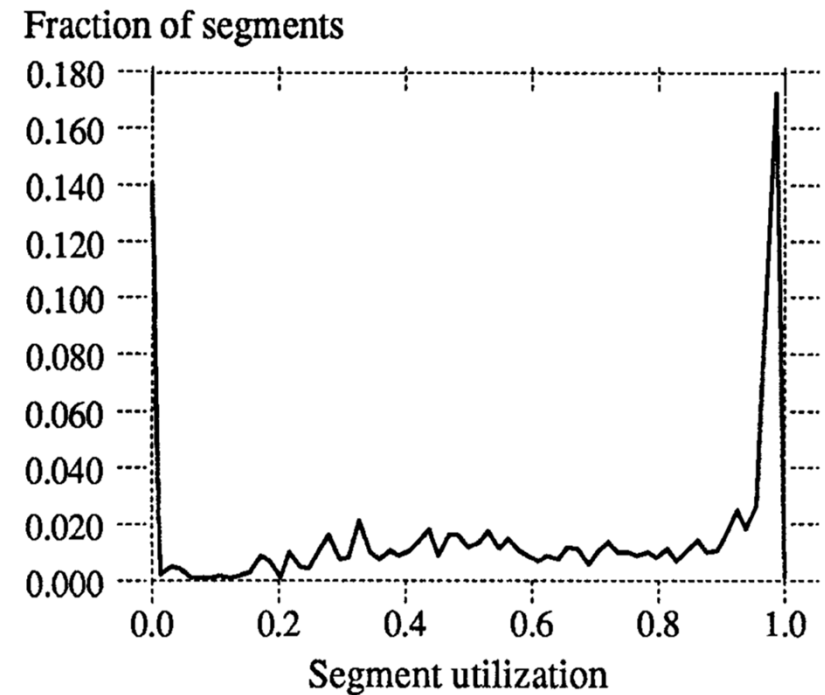
6.2 Cleaning overheads

- LFS의 cleaning은 쓰기 증폭을 발생시킴

→ Cleaning은 항상 수행하는 것이 아닌 Disk가 어느 정도 찬 후에 실행하기 때문에 성능 저하가 거의 없음

- LFS의 성능이 더 좋게 나타나는 이유는 Hot/cold data가 잘 분리되어 있기 때문

→ Cold data는 수정이 잘 되지 않기 때문에 실제 cleaning 비용은 생각보다 작고 감당 가능



실제 Sprite LFS 사용에서 관측된 utilization 분포

6.3 Crash recovery

- FFS는 crash가 발생할 경우, 전체 File system check (fsck)를 수행
→ Disk의 크기가 클 수록 복구 시간이 늘어난다
- LFS는 checkpoint → Roll-forward만 수행하기 때문에 FFS 대비 Recovery 시간이 낮다
- Checkpoint에 최신 inode map의 위치가 기록되어 있기 때문에 checkpoint를 한 번 읽고 최근 segment 만 검사하면 된다.

6.4 Other overheads in Sprite LFS

1. LFS는 최신 inode map을 저장할 추가적인 memory 공간이 필요하다
→ Inode map의 크기가 작기 때문에 memory 내에 충분히 저장 가능
2. Segment summary block을 저장할 Disk 공간이 필요하다
→ Segment summary block 역시 segment 대비 크기가 작기 때문에 overhead를 무시할 수 있다
3. Checkpoint를 주기적으로 기록하기 때문에 생기는 오버헤드
→ 어차피 몇 초 ~ 몇 십초에 한 번 정도 수행하기 때문에 성능 영향 거의 없음
4. Cleaner가 background에서 계속 실행되면서 CPU와 disk를 사용한다
→ 여기서 cleaner가 idle 시간에 수행되도록 설계해서 사용자가 느끼는 성능 저하가 거의 없다

7. Conclusion

- 기존 FS의 문제점 = seek 때문에 CPU의 뛰어난 성능을 제대로 활용하지 못 하고 있다.
- 그러나 LFS는 랜덤 쓰기를 순차 쓰기로 바꾸었고, cost-benefit을 잘 활용하면 cleaning 비용이 작고 효율적임을 보여주었다.
- 이를 통해 LFS는 기존 FS 대비 실제 성능이 향상된 것을 확인

Thank you